OOP_Thinking_ChatGPT

May 21, 2026

1 OOP Thinking

Prompt for ChatGPT-5.5 Plus:

I'm new to object-oriented coding in python. I want to learn more using jupyter notebook examples and guidance on the underlying thinking and reasoning. I have a pretty good grasp of python in general but I'm not trained in object-oriented thinking. Set some goals and give me some meaningful exercises.

You already have a strong advantage: you know Python and you seem comfortable with technical subjects (DSP, decoding algorithms, embedded systems, EE teaching). The gap is probably **not syntax** — it is learning to think in terms of:

> *What objects exist? What state do they own? What behavior belongs with that state? What responsibilities should be grouped together?*

Most people learn OOP backwards by memorizing `class`, `__init__`, inheritance, etc. I'd suggest learning it in layers.

1.1 Phase 1 — Learn object thinking (Week 1)

1.1.1 Goal 1: Stop thinking "functions first"

Instead of:

```
filter_signal(x)
compute_fft(x)
plot_signal(x)
```

Ask:

```
sig = Signal(x)

sig.filter()
sig.fft()
sig.plot()
```

Question:

What owns the data?

The answer often becomes the object.

1.1.2 Exercise 1: Build a `Signal` object

Notebook: `01_signal_class.ipynb`

Start with:

```
import numpy as np
import matplotlib.pyplot as plt

class Signal:

    def __init__(self, samples, fs):
        self.samples = np.array(samples)
        self.fs = fs

    def duration(self):
        return len(self.samples) / self.fs

    def energy(self):
        return np.sum(self.samples**2)

    def plot(self):

        t = np.arange(len(self.samples))/self.fs

        plt.plot(t, self.samples)
        plt.xlabel("Time")
        plt.ylabel("Amplitude")
        plt.show()
```

Try:

```
x = Signal(
    samples=np.sin(np.linspace(0,10,200)),
    fs=100
)

print(x.duration())
print(x.energy())

x.plot()
```

1.1.3 Thinking exercise

Ask:

- Why is `fs` stored?
- Why is `energy()` inside the object?
- Should plotting belong here?
- What should remain external?

Write answers in notebook markdown cells.

1.2 Phase 2 — State and behavior (Week 2)

Objects exist because **state persists**.

1.2.1 Exercise 2: Oscilloscope simulator

Create:

```
class Oscilloscope:
```

Internal state:

```
sample_rate
voltage_scale
time_scale
current_signal
```

Methods:

```
load_signal()
autoscale()
measure_rms()
display()
```

Example:

```
scope = Oscilloscope()

scope.load_signal(sig)

scope.autoscale()

scope.display()
```

Notebook questions:

1. Which variables belong to object state?
2. Which should be method arguments?
3. What happens if two scopes observe different signals?

1.3 Phase 3 — Multiple interacting objects

Most real OOP begins here.

1.3.1 Exercise 3: Communication chain

Build:

```
Source
Encoder
Channel
Decoder
Receiver
```

Flow:

```
src = Source()

enc = Encoder()

ch = Channel()

dec = Decoder()

rx = Receiver()

bits = src.generate()

coded = enc.encode(bits)

noisy = ch.transmit(coded)

decoded = dec.decode(noisy)

rx.evaluate(bits, decoded)
```

This mirrors systems work and DSP pipelines.

Ask:

- Which object owns noise power?
- Should BER live in Receiver?
- Should channel know modulation?

1.4 Phase 4 — Encapsulation and hidden internals

Most beginners expose everything.

Bad:

```
sig.samples = []
sig.fs = -10
```

Better:

```
class Signal:

    def __init__(self,samples,fs):
```

```
        self._samples = samples
        self._fs = fs

    @property
    def fs(self):
        return self._fs
```

Exercise:

Prevent illegal sample rate:

```
sig.fs = -5
```

Raise:

```
ValueError
```

1.5 Phase 5 — Inheritance (later, not first)

Many tutorials start here.

Do **not**.

Learn composition first.

Instead of:

```
class BPSKSignal(Signal)
class QPSKSignal(Signal)
```

Try:

```
class Modulator:

    def __init__(self, scheme):
        self.scheme = scheme
```

Inheritance only when:

> "A thing truly IS another thing"

Example:

```
Signal
       AnalogSignal
       DigitalSignal
```

1.6 Phase 6 — Final project (important)

Build a mini communications simulator:

Classes:

```
BitSource
ConvolutionalEncoder
BPSKModulator
AWGNChannel
ViterbiDecoder
BERCounter
Simulation
```

Run:

```
sim = Simulation()

sim.run()

sim.plot_results()
```

Given your previous work with Viterbi decoding, this will feel natural and show where OOP becomes useful.

---

2 Suggested learning milestones

Stage	Concept	Deliverable
1	Objects and state	`Signal` class
2	State ownership	`Oscilloscope`
3	Collaboration	Communication chain
4	Encapsulation	Protected members + validation
5	Composition	Modulator framework
6	Larger architecture	Full simulator
7	Advanced	Abstract classes, dataclasses, patterns

2.1 Jupyter habit I strongly recommend

For every notebook:

2.1.1 Cell 1

```
# OBJECT MAP

Signal
    state:
        samples
        fs

    behavior:
        energy()
```

```
plot()
fft()
```

2.1.2 Cell 2

Write:

> What owns the data?

2.1.3 Cell 3

Write:

> Which object is responsible?

This trains OOP thinking much faster than coding alone.

2.2 Stretch exercise

Take an existing procedural script you already have (FFT demo, filter analysis, Viterbi example, Arduino serial monitor, BER simulation).

Refactor:

Procedural:

```
generate()
encode()
channel()
decode()
measure()
```

OOP:

```
Simulation
Source
Encoder
Channel
Decoder
Metrics
```

Then answer:

1. What state moved into objects?
2. What became easier?
3. What became harder?
4. Did coupling improve?

That reflection is where object-oriented intuition develops.

If you want, next I can lay out a **10-notebook sequence with increasing difficulty and complete notebook templates**.

[ ]: